UNIDAD 1

Programación avanzada en SQL – transacciones.

TEMA 2:

Programación en SQL.

Descripción:

En esta unidad, dedicada a la **Programación Avanzada en SQL – Transacciones**, los estudiantes conocerán los fundamentos avanzados de **SQL** aplicados a la gestión de bases de datos, se estudiará el rol del **Administrador** de **Bases de Datos**, el concepto de transacción y las propiedades **ACID**, esenciales para garantizar seguridad e integridad en la información.

Además, se revisarán mecanismos de control de bloqueos, niveles de aislamiento, índices y sentencias avanzadas.

El propósito es que el estudiante adquiera las habilidades necesarias para implementar transacciones seguras y diseñar consultas optimizadas, aplicables a proyectos en Ciencia de Datos e Inteligencia Artificial.

Administración de bases de datos.

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 2. Programación en SQL. 2
2.1. Vistas y Funciones. 2
2.2. Procedimientos almacenados. 6
2.3. Triggers. 9
2.4. Optimización de consultas, Ejercicios de Aplicación. 13
Bibliografía 19

Tema 2. Programación en SQL.

La programación en SQL permite ampliar las capacidades de un sistema de gestión de bases de datos, incorpora estructuras que van más allá de las consultas básicas. A través de vistas, funciones, procedimientos almacenados y triggers, es posible encapsular lógica de negocio, automatizar tareas y optimizar el acceso a la información, estas herramientas proporcionan mayor seguridad, reutilización de código y eficiencia en la gestión de datos (Gupta & Ramachandra, 2021).

2.1. Vistas y Funciones.

Vistas.

Una **vista** es una tabla virtual que se construye a partir de una o varias tablas reales mediante una consulta SQL, su objetivo es simplificar el acceso a la información, ocultar la complejidad de las consultas y mejorar la seguridad, ya que permite restringir las columnas o filas visibles para ciertos usuarios, las vistas son muy útiles para reutilizar consultas frecuentes, estandarizar reportes y proporcionar diferentes perspectivas de los datos sin duplicarlos físicamente, aunque no almacenan datos por sí mismas, pueden actualizarse si cumplen determinadas condiciones (Silberschatz, 2021).

Ejemplo:

Código SQL

```sql
-- Crear una vista para mostrar solo pedidos activos con datos del
cliente
CREATE VIEW Vista_PedidosActivos AS
SELECT c.nombre, p.id_pedido, p.fecha, p.monto
FROM clientes c
JOIN pedidos p ON c.cliente_id = p.cliente_id
WHERE p.estado = 'ACTIVO';

-- Consultar la vista como si fuera una tabla
SELECT * FROM Vista_PedidosActivos;
```

En el ejemplo, la vista Vista_PedidosActivos simplifica una consulta frecuente y ofrece una capa de seguridad, ya que los usuarios acceden solo a la información permitida.

Vistas Actualizables.

Una **vista actualizable** es aquella que permite realizar operaciones de modificación de datos, como INSERT, UPDATE o DELETE, directamente sobre ella, reflejándose los cambios en la tabla base de la que proviene, para que esto sea posible, la vista debe construirse sobre una sola tabla y no incluir elementos que alteren la correspondencia directa con sus filas, como agregaciones, subconsultas o uniones (JOIN) (Silberschatz, 2021). Este tipo de vistas facilita el acceso restringido a los datos sin perder capacidad de edición.

Ejemplo:

Código SQL

```sql
-- Crear una vista sencilla de clientes
CREATE VIEW Vista_Clientes AS
SELECT cliente_id, nombre, ciudad
FROM clientes;

-- Insertar un nuevo cliente a través de la vista
INSERT INTO Vista_Clientes (cliente_id, nombre, ciudad)
VALUES (3001, 'Ana Torres', 'Quito');

-- Actualizar la ciudad de un cliente desde la vista
UPDATE Vista_Clientes
SET ciudad = 'Guayaquil'
WHERE cliente_id = 3001;

-- Consultar la vista
SELECT * FROM Vista_Clientes;
```

En el ejemplo, la vista Vista_Clientes es actualizable porque se basa en una sola tabla (clientes), sin funciones de agregación, JOINs ni subconsultas complejas, esto permite realizar operaciones INSERT, UPDATE y DELETE directamente sobre la vista, reflejándose en la tabla original.

Funciones.

Las **funciones** son rutinas programadas que permiten encapsular un conjunto de instrucciones bajo un nombre definido mediante CREATE FUNCTION, pueden recibir uno o más parámetros de entrada, ejecutar operaciones de cálculo o procesamiento y devolver un resultado. Los resultados puede ser un valor único, en el caso de las funciones escalares, o un conjunto de filas, en el caso de las funciones de tabla.

Funciones escalares: Retornan un solo valor (por ejemplo, convertir texto en mayúsculas o calcular un promedio).

Ejemplo:

Código SQL

```sql
-- Crear una función escalar para calcular el descuento del 10%
CREATE FUNCTION CalcularDescuento (@monto DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @monto * 0.10;
END;

-- Uso de la función en una consulta
SELECT id_pedido, monto,
       dbo.CalcularDescuento(monto) AS Descuento
FROM pedidos;
```

La función CalcularDescuento recibe como parámetro el valor del pedido y devuelve el descuento del 10%. Al usarla en una consulta,

se puede calcular el descuento de cada registro de manera automática y uniforme.

Funciones de tabla: Devuelven un conjunto de filas que pueden usarse como si fueran una tabla en una consulta.

Ejemplo:

Código SQL

```
-- Crear una función de tabla para obtener los pedidos de un cliente
CREATE FUNCTION PedidosPorCliente (@cliente_id INT)
RETURNS TABLE
AS
RETURN (
    SELECT id_pedido, fecha, monto, estado
    FROM pedidos
    WHERE cliente_id = @cliente_id
);

-- Uso de la función en una consulta
SELECT *
FROM dbo.PedidosPorCliente(101);
```

La función PedidosPorCliente recibe un cliente_id y devuelve todos los pedidos asociados a ese cliente. Al invocarla, puede integrarse en consultas más complejas, como si fuera una tabla real.

Las funciones permiten encapsular lógica repetitiva, simplificar consultas complejas y mejorar la reutilización del código en la administración de bases de datos (Foufoulas & Simitsis, 2023).

Parámetros.

Los **parámetros** son variables que se utilizan para enviar valores de entrada a una función o procedimiento en SQL, actúan como contenedores que reciben datos específicos al momento que la función

es invocada, los parámetros hacen que las funciones sean más flexibles y reutilizables, ya que el mismo bloque de código puede ejecutarse con distintos valores de entrada, produce resultados personalizados según cada caso (Silberschatz, 2021).

DBMS	Sintaxis de parámetros	Ejemplo de función escalar

SQL Server (T-SQL)	Usa @ antes del nombre del parámetro.	sql
		CREATE FUNCTION CalcularIVA (@monto DECIMAL(10,2))
		RETURNS DECIMAL(10,2)
		AS
		BEGIN
		 RETURN @monto * 0.12;
		END;

PostgreSQL (PL/pgSQL)	No usa @, se define directamente.	sql
		CREATE FUNCTION calculariva(monto NUMERIC)
		RETURNS NUMERIC AS $$
		BEGIN
		 RETURN monto * 0.12;
		END;
		$$ LANGUAGE plpgsql;

MySQL / MariaDB	No usa @ en parámetros; @ es para variables de usuario.	sql CREATE FUNCTION CalcularIVA(monto DECIMAL(10,2)) RETURNS DECIMAL(10,2) DETERMINISTIC RETURN monto * 0.12;

Oracle (PL/SQL)	No usa @, define parámetros con nombre directo.	sql
		CREATE OR REPLACE FUNCTION CalcularIVA (monto NUMBER)
		RETURN NUMBER IS
		BEGIN
		 RETURN monto * 0.12;
		END;

2.2. Procedimientos almacenados.

Un **procedimiento almacenado** es un conjunto de instrucciones SQL que se guarda en el servidor de base de datos y puede ejecutarse al momento que sea necesario, a diferencia de las funciones, que siempre devuelven un valor o conjunto de resultados, los procedimientos almacenados pueden realizar múltiples operaciones: insertar, actualizar, eliminar, validar reglas de negocio o incluso llamar a otros procedimientos y funciones (Silberschatz, 2021).

Su uso mejora la **seguridad**, porque encapsula la lógica en el servidor, y aumenta la **eficiencia**, al evitar enviar múltiples consultas desde la aplicación, favorece la **reutilización del código** y la **estandarización de procesos** dentro de una organización.

Ejemplo:

Código SQL

```
-- Crear un procedimiento almacenado para actualizar saldo
CREATE OR REPLACE PROCEDURE ActualizarSaldo(cliente INT, valor
NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE cuentas
    SET saldo = saldo + valor
    WHERE cliente_id = cliente;

    -- Validación de saldo negativo
    IF (SELECT saldo FROM cuentas WHERE cliente_id = cliente) < 0
THEN
        RAISE NOTICE 'El saldo no puede ser negativo';
        ROLLBACK;
    END IF;
END;
$$;

-- Ejecutar el procedimiento
CALL ActualizarSaldo(101, -500);
```

El procedimiento InsertarPedido encapsula la lógica para registrar pedidos, reduce errores y centraliza la operación en el servidor.

Ejemplo:

Procedimiento almacenado con validación

Código SQL

```
-- Crear un procedimiento almacenado para actualizar saldo
CREATE OR REPLACE PROCEDURE ActualizarSaldo(cliente INT, valor
NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE cuentas
    SET saldo = saldo + valor
    WHERE cliente_id = cliente;

    -- Validación de saldo negativo
    IF (SELECT saldo FROM cuentas WHERE cliente_id = cliente) < 0
THEN
        RAISE NOTICE 'El saldo no puede ser negativo';
        ROLLBACK;
    END IF;
END;
$$;

-- Ejecutar el procedimiento
CALL ActualizarSaldo(101, -500);
```

El procedimiento controla la actualización del saldo y valida que no se vuelva negativo.

2.3. Triggers.

Un **trigger** es un programa almacenado que se ejecuta automáticamente en respuesta a eventos **INSERT**, **UPDATE** o **DELETE** sobre una tabla o vista, puede ser **BEFORE/INSTEAD OF** (antes/sustituye la operación) o **AFTER** (después), y a nivel **fila** o **sentencia**, según el DBMS. Sirve para **hacer cumplir reglas de negocio**, **auditar cambios**, **mantener integridad derivada** y **sincronizar** datos entre tablas, sin requerir código en la aplicación (Connolly & Begg, 2021).

Uso adecuado de triggers en bases de datos

Los **triggers** deben emplearse si se necesita que ciertas acciones se ejecuten automáticamente al ocurrir un evento en la base de datos, sin depender de la aplicación (Silberschatz, 2021).

Son especialmente útiles en tres escenarios:

1. **Auditoría de cambios**: Registrar quién, cuándo y qué valores se modificaron en una tabla (por ejemplo, cambios de estado en un pedido).
2. **Validación de reglas de negocio**: Garantizar que se cumplan condiciones que no pueden controlarse solo con restricciones simples, como evitar saldos negativos o impedir la eliminación de un registro crítico.
3. **Mantenimiento automático de datos relacionados**: Actualizar totales, recalcular saldos o sincronizar información en tablas dependientes cada vez que ocurre un cambio.

Sin embargo, no deben usarse para lógica muy compleja ni para reemplazar restricciones (PRIMARY KEY, FOREIGN KEY, CHECK) que ya ofrece el sistema gestor, tampoco es recomendable abusar de ellos, ya

ACREDITADA · QS World University Rankings Latin America · THE World University Rankings 2023 Latin America · THE Impact Summit · soludable

que pueden afectar el rendimiento y dificultar el seguimiento de errores si se encadenan demasiados triggers.

Ejemplo:

Código SQL

```
-- Crear un trigger que muestre un mensaje al insertar un nuevo
cliente
CREATE TRIGGER trg_MensajeNuevoCliente
ON clientes
AFTER INSERT
AS
BEGIN
    PRINT 'Se ha registrado un nuevo cliente en la base de datos.';
END;
```

Cada vez que se inserta un nuevo registro en la tabla clientes, el trigger se activa y muestra un mensaje, el ejemplo ayuda a ver cómo un trigger responde a un evento (AFTER INSERT).

Código SQL

```
-- Crear tabla de auditoría
CREATE TABLE auditoria_clientes (
    id SERIAL PRIMARY KEY,
    cliente_id INT,
    accion VARCHAR(20),
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Función asociada al trigger
CREATE OR REPLACE FUNCTION fn_auditar_clientes()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO auditoria_clientes (cliente_id, accion)
    VALUES (NEW.cliente_id, TG_OP); -- TG_OP indica si fue INSERT,
UPDATE o DELETE
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Crear trigger que llama a la función
CREATE TRIGGER trg_auditoria_clientes
AFTER INSERT ON clientes
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_clientes();
```

Cada vez que se inserta un cliente, se guarda en la tabla auditoria clientes el ID del cliente, la acción y la fecha.

Ejemplo:

Código SQL

```sql
-- Crear tabla de auditoría
CREATE TABLE auditoria_pedidos (
    id_auditoria INT IDENTITY(1,1) PRIMARY KEY,
    id_pedido INT,
    estado_anterior VARCHAR(20),
    estado_nuevo VARCHAR(20),
    fecha DATETIME DEFAULT GETDATE()
);

-- Crear trigger para registrar cambios en el estado de un pedido
CREATE TRIGGER trg_AuditoriaEstadoPedido
ON pedidos
AFTER UPDATE
AS
BEGIN
    INSERT INTO auditoria_pedidos (id_pedido, estado_anterior,
estado_nuevo)
    SELECT d.id_pedido, d.estado, i.estado
    FROM deleted d
    JOIN inserted i ON d.id_pedido = i.id_pedido
    WHERE d.estado <> i.estado;
END;
```

Al actualizarse el **estado** de un pedido, el trigger registra el valor anterior (deleted) y el nuevo (inserted), se crea un historial automático que permite rastrear cambios sin depender de la aplicación.

Ejemplo:

Código SQL

```sql
-- Crear tabla para guardar registros eliminados
CREATE TABLE auditoria_eliminados (
    id_auditoria INT IDENTITY(1,1) PRIMARY KEY,
    id_cliente INT,
    nombre VARCHAR(100),
    fecha_eliminacion DATETIME DEFAULT GETDATE()
);

-- Trigger que guarda clientes eliminados
CREATE TRIGGER trg_ClientesEliminados
ON clientes
AFTER DELETE
AS
BEGIN
    INSERT INTO auditoria_eliminados (id_cliente, nombre)
    SELECT d.cliente_id, d.nombre
    FROM deleted d;
END;
```

El trigger guarda en auditoria_eliminados la información del cliente borrado, queda disponible para auditorías futuras.

2.4. Optimización de consultas, Ejercicios de Aplicación.

Optimización de consultas.

La **optimización de consultas** es el proceso mediante el cual un sistema gestor de bases de datos elige la forma más eficiente de ejecutar una instrucción SQL, una consulta mal diseñada puede consumir demasiados recursos, tardar en responder o bloquear otras operaciones (Rahman et al., 2024).

Entre las principales estrategias de optimización están:

1. **Uso de índices adecuados**: Aprovechar estructuras que aceleran búsquedas.
2. **Selección precisa de columnas y filas**: Evitar SELECT *.
3. **JOIN bien planteados**: Usar las claves correctas y el orden lógico de combinación.
4. **Evitar subconsultas innecesarias**: Reemplazarlas por JOIN o CTE si corresponde.
5. **Análisis de planes de ejecución**: Revisar cómo el motor procesa la consulta para detectar cuellos de botella.

Con estas prácticas se logra reducir el consumo de memoria y tiempo de CPU, mejora el rendimiento global de la base de datos.

Ejemplo:

Código SQL

```sql
-- Consulta poco eficiente
SELECT *
FROM pedidos p, clientes c
WHERE p.cliente_id = c.cliente_id
AND p.monto > 500;

-- Consulta optimizada
SELECT c.nombre, p.id_pedido, p.monto
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.cliente_id
WHERE p.monto > 500;
```

La segunda versión es más eficiente porque: Selecciona solo las columnas necesarias; usa sintaxis explícita de JOIN; permite al optimizador aplicar índices en monto y cliente_id.

Ejercicio de Aplicación

En una empresa se desea conocer a los Clientes con mayor gasto que el promedio, para esta tarea de debe utilizar índices, CTE y JOIN. De manera opcional puede utilizar funciones de ventana).
Para la resolución del ejercicio, se dispones de las tablas clientes y pedidos, con esto se debe: Crear índices apropiados para acelerar búsquedas por cliente/estado/fecha; calcular el gasto total por cliente solo para pedidos ACTIVOS; obtener la lista de clientes cuyo total supera el promedio general y asignar un ranking por monto total.

Código SQL Parte 1

```sql
/* ------------------------
   0. Esquema y datos de ejemplo
   ------------------------ */
-- Clientes
DROP TABLE IF EXISTS clientes;
CREATE TABLE clientes (
  cliente_id   INT PRIMARY KEY,
  nombre       VARCHAR(100) NOT NULL
);

-- Pedidos
DROP TABLE IF EXISTS pedidos;
CREATE TABLE pedidos (
  id_pedido    INT PRIMARY KEY,
  cliente_id   INT NOT NULL,
  fecha        DATE NOT NULL,
  monto        NUMERIC(12,2) NOT NULL,
  estado       VARCHAR(20) NOT NULL,
  FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
);

-- Cargar datos
INSERT INTO clientes (cliente_id, nombre) VALUES
(101,'Ana Torres'), (102,'Bruno Díaz'), (103,'Carla Méndez'),
(104,'Diego Rojas'), (105,'Elisa Cruz');

INSERT INTO pedidos (id_pedido, cliente_id, fecha, monto, estado)
VALUES
(1,101,'2024-01-10',120.00,'ACTIVO'),
(2,101,'2024-02-14',380.00,'ACTIVO'),
(3,102,'2024-02-20',150.00,'ANULADO'),
(4,102,'2024-03-03',600.00,'ACTIVO'),
(5,103,'2024-03-15',220.00,'ACTIVO'),
(6,103,'2024-03-30',180.00,'ACTIVO'),
(7,104,'2024-04-05',900.00,'ACTIVO'),
(8,104,'2024-04-18',150.00,'ACTIVO'),
(9,105,'2024-04-25',80.00 ,'ACTIVO');
```

Código SQL Parte 2

```
/* -------------------------
   1. Índices recomendados
   ------------------------- */
-- SQL Server:
-- Crea índice compuesto y "cubriente" para filtros/orden por cliente,
estado, fecha
-- (INCLUDE deja el monto disponible sin buscar en la tabla)
-- CREATE NONCLUSTERED INDEX IX_Pedidos_ClienteEstadoFecha
-- ON dbo.pedidos (cliente_id, estado, fecha) INCLUDE (monto);

-- PostgreSQL (11+): índice con columnas INCLUDE (no clave)
-- CREATE INDEX idx_pedidos_cliente_estado_fecha
-- ON pedidos (cliente_id, estado, fecha) INCLUDE (monto);

-- Alternativa filtrada/partial index (si la mayoría de pedidos NO están
activos):
-- SQL Server (filtered index):
-- CREATE NONCLUSTERED INDEX IX_Pedidos_Activos
-- ON dbo.pedidos (cliente_id, fecha) INCLUDE (monto) WHERE estado =
'ACTIVO';
--
-- PostgreSQL (partial index):
-- CREATE INDEX idx_pedidos_activos
-- ON pedidos (cliente_id, fecha) INCLUDE (monto) WHERE estado = 'ACTIVO';

/* -------------------------
   2. Consulta principal con CTE
   ------------------------- */
WITH total_por_cliente AS (
  SELECT  p.cliente_id,
          SUM(p.monto) AS total
  FROM pedidos p
  WHERE p.estado = 'ACTIVO'
  GROUP BY p.cliente_id
),
promedio AS (
  SELECT AVG(total) AS promedio_general
  FROM total_por_cliente
)
SELECT  c.nombre,
        t.total
FROM total_por_cliente t
JOIN promedio pr ON 1=1          -- CROSS JOIN implícito
JOIN clientes c  ON c.cliente_id = t.cliente_id
WHERE t.total > pr.promedio_general
ORDER BY t.total DESC;
```

Código SQL Parte 3

```sql
/* ------------------------
   3. (Opcional) Ranking con función de ventana
   ------------------------ */
SELECT  c.nombre,
        SUM(p.monto) AS total,
        RANK() OVER (ORDER BY SUM(p.monto) DESC) AS posicion
FROM pedidos p
JOIN clientes c ON c.cliente_id = p.cliente_id
WHERE p.estado = 'ACTIVO'
GROUP BY c.nombre
ORDER BY total DESC;
```

Explicación desarrollo ejercicio.

1. Datos base y llaves
    a. Se crean tablas normalizadas (clientes, pedidos) con PK y FK, y se cargan registros de ejemplo, esto permite probar filtros por cliente_id, por estado y por rango de fecha.
2. Índices
    a. El índice compuesto (cliente_id, estado, fecha) responde a los patrones de consulta: filtrar por cliente y estado, y eventualmente ordenar por fecha.
    b. La cláusula INCLUDE (monto) vuelve al índice "de cobertura": el motor puede resolver la consulta sin volver a la tabla, reduce lecturas.
    c. En escenarios con muchos pedidos no activos, un índice filtrado/partial sobre estado='ACTIVO' reduce tamaño y mejora selectividad.
3. CTE total_por_cliente
    a. Resume en una tabla temporal lógica el total por cliente solo con pedidos ACTIVOS. Mejora la legibilidad al separar el cálculo del resto de la consulta.

4. CTE promedio
    a. Calcula el promedio general de gasto por cliente. Se hace un JOIN (equivalente a CROSS JOIN) para comparar cada total contra ese promedio.
5. Consulta final (JOIN + filtro)
    a. Se vinculan los totales con clientes para mostrar el nombre y se filtra t.total > promedio_general.
    b. El ORDER BY permite listar primero a quienes más gastan. Con el índice propuesto, el optimizador puede usar Index Seek/Index Scan eficiente.
6. (Opcional) Función de ventana RANK()
    a. Agrega posiciones sin perder el detalle del total por cliente. Es ideal para reportes (top N, percentiles, etc.).
7. Qué aprende el estudiante
    a. Diseñar índices según los filtros reales.
    b. Escribir consultas limpias con CTE y JOIN.
    c. Medir/interpretar mejoras: menos lecturas y mejor tiempo de ejecución (ver plan con EXPLAIN/SHOWPLAN).

Bibliografía

Barthels, C., Müller, I., Taranov, K., Alonso, G., & Hoefler, T. (2019). Strong consistency is not hard to get: Two-phase locking and two-phase commit on thousands of cores. *Proceedings of the VLDB Endowment*, *12*(13), 2325-2338. https://doi.org/10.14778/3358701.3358702

Beynon-Davies, P. (2021). Data and Database Administration. En P. Beynon-Davies (Ed.), *Database Systems* (pp. 129-143). Macmillan Education UK. https://doi.org/10.1007/978-1-349-13722-0_11

Bouajjani, A., Enea, C., & Román-Calvo, E. (2025). On the Complexity of Checking Mixed Isolation Levels for SQL Transactions. En R. Piskac & Z. Rakamarić (Eds.), *Computer Aided Verification* (Vol. 15934, pp. 315-337). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-98685-7_15

Connolly, T. M., & Begg, C. E. (2021). *Database systems: A practical approach to design, implementation, and management* (4th ed). Addison-Wesley.

Elmasri, R., & Navathe, S. (2021). *Fundamentals of database systems* (Seventh edition). Pearson.

Foufoulas, Y., & Simitsis, A. (2023). Efficient Execution of User-Defined Functions in SQL Queries. *Proceedings of the VLDB Endowment*, *16*(12), 3874-3877. https://doi.org/10.14778/3611540.3611574

Gupta, S., & Ramachandra, K. (2021). Procedural extensions of SQL: Understanding their usage in the wild. *Proceedings of the VLDB Endowment*, *14*(8), 1378-1391. https://doi.org/10.14778/3457390.3457402

Jorge Santiago Nolasco Valenzuela. (2018). *Python Aplicaciones prácticas*. RA-MA Editorial.

Leis, V., Kundhikanjana, K., Kemper, A., & Neumann, T. (2020). Efficient processing of window functions in analytical SQL queries. *Proc. VLDB Endow.*, *8*(10), 1058-1069. https://doi.org/10.14778/2794367.2794375

Özsu, M. T. (2022). Database Administrator (DBA). En *Encyclopedia of Database Systems* (pp. 935-936). Springer, New York, NY. https://doi.org/10.1007/978-1-4614-8265-9_80652

Rahman, M. M., Islam, S., Kamruzzaman, M., & Joy, Z. H. (2024). Advanced query optimization in SQL databases for real-time big data analytics.

Academic Journal on Business Administration, Innovation & Sustainability, *4*(3), 1-14.

Rodríguez Guerrero, R., Vanegas, C. A., & Castang Montiel, G. (2020). *Python a su alcance*. Editorial Universidad Distrital Francisco José de Caldas. Editorial UD.

Roller, J. (2024, julio 8). Ensuring Data Integrity: The Role of ACID Transactions in Modern Distributed Microservices Architecture. *IEEE Computer Society*. https://www.computer.org/publications/tech-news/community-voices/acid-transactions-in-distributed-microservices-architecture/

Shin, Y., Ahn, J., & Im, D.-H. (2022). Join optimization for inverted index technique on relational database management systems. *Expert Systems with Applications*, *198*, 116956. https://doi.org/10.1016/j.eswa.2022.116956

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (5th ed). McGraw-Hill Higher Education.

Silberschatz, A. (with Korth, H. F., Sudarshan, S., Sáenz Pérez, F., García Cordero, A., Correas Fernández, J., & Grau Fernández, L.). (2021). *Fundamentos de bases de datos* (5a ed). McGraw-Hill.

Stuparu, D., & Petrescu, M. (2025). Common Table Expression: Different database systems approach. *ResearchGate*. https://www.researchgate.net/publication/259441876_Common_Table_Expression_Different_database_systems_approach

Wanigsoriya, S. (2024). Database Administration Developments and Research in Contemporary Digital Environments. *ResearchGate*. https://doi.org/10.13140/RG.2.2.25897.81763